

A Mathematical Model for the Flexible Job Shop Scheduling Problem With Availability Constraints

Tom Perroux, Taha Arbaoui, Leila Merghem-Boulahia

*Laboratory of Computer Science and Digital Society, University of
Technology of Troyes, 12 Rue Marie Curie, 10000, Troyes, France
(e-mails: tom.perroux@utt.fr, taha.arbaoui@utt.fr,
leila.merghem-boulahia@utt.fr).*

Abstract: We tackle the Flexible Job Shop Problem (FJSSP) subject to availability constraints due to maintenance tasks on machines. Previous studies explored meta-heuristic methods to address this problem and solved small-sized instances generated from classical FJSSP benchmark. We first introduce a new benchmark, comprising more than 2000 instances, for the problem based on the well-known FJSSP benchmarks. A mixed integer model is then developed to solve this problem. We carry out a computational study on the proposed benchmark and analyze its performances. Results show that our model can solve small and medium size instances efficiently. We also observe important variation in complexity depending on the class of instance which makes the proposed benchmark interesting to evaluate our approach and further improve the solving process in future works.

Copyright © 2023 The Authors. This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0/>)

1. INTRODUCTION

The Job Shop Scheduling Problem (JSSP) is a well known problem of combinatorial optimisation. JSSP consists of a set of jobs where each job is a sequence of operations that must be scheduled on a set of machines. Each operation requires exactly one machine such that when an operation is being processed on a machine, no other operation can be processed on that machine until the operation is done. The ordering of the operations has a significant impact on the quality of the solution. To measure the quality of the solution, several performance indicators have been studied. The most common indicators are the makespan minimisation and the tardiness minimisation. There has been a lot of studies on how to solve the JSSP in the literature especially since the work of Taillard (1993) in which a large benchmark for this problem has been proposed.

In real-world industrial applications, it is common to have multi-purpose machines, such that each operation can be scheduled on a restricted set of machines. The Flexible Job Shop Scheduling Problem (FJSSP) was first introduced by Brucker and Schlie (1990) to model job-shop problems with this property. In FJSSP, operations can only be scheduled on available machines which is a sub-set of the set of machine, thus the FJSSP is a generalisation of the JSSP. Since the JSSP is a NP-hard problem, the FJSSP is also NP-hard and is even more difficult to solve. The FJSSP can be decomposed into two sub-problems: an assignment sub-problem, which consists in assigning each operation to a machine out of a set of eligible machines, and a scheduling sub-problem, which consists in sequencing the operations on the selected machines in order to optimize the objective value.

Tabu search was among the first successful methods to solve the FJSSP with the work of Brandimarte (1993), Hurink et al. (1994), Dauzère-Pères and Paulli (1997), Barnes and Chambers (1995). Kacem et al. (2002) and Fattahi et al. (2007) proposed a large number of instances that represent the current reference benchmark for this problem. Behnke and Geiger (2012) later proposed a review of the common instances for the FJSSP, solved it by mean of constraint programming and extended the benchmark with new instances based on work centres.

The difficulty of the problem has encouraged the use of heuristic and meta-heuristic methods to solve this problem. Pezzella et al. (2008) proved that genetic algorithms are effective to solve the FJSSP. Zhang et al. (2009) proposed an effective multi-objective hybrid particle swarm optimization. Xing et al. (2010) studied a knowledge-based ant colony optimization algorithm. Wang et al. (2012) presented an artificial bee colony algorithm for solving the FJSSP.

In real-world flexible shop problems, machines can be unavailable during certain periods of time due to setup operations, machine breakdown or maintenance operations. For the FJSSP, numerous works considered sequence-dependant setup time where the presence and the duration of a setup operation between two consecutive operations on a machine is dependant on the preceding operation and the next operation to be scheduled. Saidi-Mehrabad and Fattahi (2007) proposed a MIP model for the FJSSP with the consideration of sequence-dependant setup times and solved it with tabu search. Rossi (2014) studied an ant colony approach for this problem and additionally considered transportation times. Kress et al. (2019) proposed exact and heuristic decomposition-based approaches for the FJSSP with both worker constraints

and sequence-dependant setup time. Unavailability due to machine breakdowns are unpredictable, thus researchers explored efficient methods to obtain robust scheduling. Al-Hinai and ElMekkawy (2011) proposed a hybrid genetic algorithm and split the decision process into two stages, the first stage minimizes the makespan while the second stage minimizes a robust performance measure. Xiong et al. (2013) solved a multi-objective evolutionary algorithm where the first objective is the makespan and the second is the robustness.

Availability constraints that are known in advance such as maintenance tasks should be considered in the scheduling. Aggoune (2004) first considered availability constraints for the job-shop scheduling problem but they did not study the flexible case. The FJSSP with non-fixed availability (FJSSP-nfa) was later introduced by Gao et al. (2006), where maintenance tasks have a deterministic duration and a time interval in which it must be scheduled. In the study, they solve the FJSSP-nfa with a hybrid genetic algorithm (Hybrid GA) on a small benchmark generated from classical FJSSP instances. Wang and Yu (2010) further explored the FJSSP-nfa and considered maintenance tasks with restricted maintenance resources and proposed a filtered beam search based heuristic algorithm. Rajkumar et al. (2011) solved the FJSSP-nfa with a greedy randomised adaptive search procedure algorithm and compared their results to the literature.

Li and Pan (2012) and Li et al. (2014) presented a chemical-reaction optimization and a discrete artificial bee colony algorithm for the FJSSP-nfa. They considered potential multiple preventive maintenance tasks on machines while minimizing the makespan, the total workload and the maximum workload. They showed the efficiency of their approach by comparing their results to the literature on the same instances with different time windows. Ziaee (2014) designed an efficient heuristic algorithm based on a constructive procedure considering maintenance constraints that aims at producing reasonably good solutions very quickly that can be embedded in other solution approaches. El Khoukhi et al. (2017) considered unavailability periods on machines with fixed interval. They solved this problem with a dual-ants colony algorithm. Jing et al. (2017) studied probability based-preventive maintenance. In this problem, they consider a probability of machine failure and process a preventive maintenance if the probability reaches a specific threshold. They solved this problem with a multi-objective GA. In a more recent study, Wang et al. (2021) considered both preventive maintenance and transportation process with energy consumption. They used a multi-objective evolutionary algorithm as a solution approach.

Besides the aforementioned works, the FJSSP-nfa has not seen much interest, which can partially be explained by the lack of benchmark instances. Indeed, all the works used a limited number of instances (less than 10) which does not allow a clear and fair comparison between approaches.

Most studies with maintenance constraints considered heuristic and meta-heuristic approaches. The efficiency of current days solvers (such as IBM CPLEX) and the lack of exact approaches led us to consider a MIP solution method to close the gap in the literature. Exact solution

approaches can reach optimality on some instances which is valuable to evaluate the performance of non-exact methods such as meta-heuristics in later studies. To the best of our knowledge, no MIP model has been proposed in the literature for the FJSSP-nfa. Our first contribution is to propose a novel MIP formulation for the FJSSP-nfa. To evaluate the performance of our method we faced the lack of publicly available benchmarks for this problem with only a few instances from Gao et al. (2006) and Wang and Yu (2010). Our second contribution is to propose a large benchmark for the problem, generated from classical FJSSP benchmarks. The proposed benchmark is both diverse in terms of instance size and maintenance location on the horizon. A result comparison is performed to evaluate our approach and the impact of maintenance tasks in the scheduling process.

The remainder of the paper is organized as follows. In Section 2, the FJSSP-nfa is formally defined and a mixed integer formulation is proposed. Section 3 is devoted to describing the proposed benchmark and presenting the obtained results of the mixed integer formulation. The conclusion is to be found in Section 4.

2. THE PROPOSED MIXED INTEGER FORMULATION

2.1 Problem Definition

In the FJSSP-nfa, there are n jobs that are to be scheduled on a set of m machines (denoted as K). Each job i comprises n_i operations o_{ij} , $j \in \{1, \dots, n_i\}$ that must be scheduled in the given order, i.e. operation o_{ij} cannot start before the preceding operation $o_{i,j-1}$ is completed (operation o_{i1} has no preceding operation). Machines are multi-purpose where operation o_{ij} can be scheduled on any machine, if the machine is unoccupied, from the set A_{ij} which is the set of available machines for this operation. When a machine starts processing an operation, it will be occupied for p_{ijk} time units. The processing time p_{ijk} depends both on the operation o_{ij} and the machine k on which it is scheduled. In addition, there are L_k maintenance tasks that must be scheduled on each machine $k \in K$. Each maintenance task l_k has a corresponding time window in which it must be scheduled (a maintenance task cannot be processed before the start of the time window and must be completed before the end of the time window). The processing time of a maintenance task is $\tilde{p}_{kl}$ and no activity (operation or maintenance task) can be processed when a maintenance task is being performed. Job preemption is not allowed and the setup time between operations is considered machine-independent and is included in the processing time.

2.2 The Proposed Formulation

The parameters for the FJSSP-nfa are summarised in Table 1. In Table 2, we describe the decision variables used in the proposed mixed integer formulation for the FJSSP-nfa. The completion time of operations and maintenance tasks are determined respectively with variables c_{ij} and y_{kl} . x_{ijk} are the variables for the assignment of operations to machines. $v_{ijij'}$ and w_{ijkl} are additional variables included to decide whether an activity is scheduled before

or after another activity to avoid overlap between activities on the same machine.

The mixed integer model for the FJSSP-nfa is presented hereafter using Constraints (1)-(11). It is worth noting that this is the first mixed integer linear program proposed as previously proposed formulated are not linear.

$$\min C_{max} \quad (1)$$

$$C_{max} \geq c_{ij}, \quad \forall i \in I, j \in O_i \quad (2)$$

$$\sum_{k \in A_{ij}} x_{ijk} = 1, \quad \forall i \in I, j \in O_i \quad (3)$$

$$c_{ij} - c_{i(j-1)} \geq \sum_{k \in A_{ij}} x_{ijk} p_{ijk}, \quad \forall i \in I, j \in O_i \quad (4)$$

$$y_{kl} - \tilde{p}_{kl} \geq t_{kl}^E, \quad \forall k \in K, l \in L_k \quad (5)$$

$$y_{kl} \leq t_{kl}^L, \quad \forall k \in K, l \in L_k \quad (6)$$

$$c_{i'j'} \geq c_{ij} + \sum_{k \in A_{i'j'}} p_{i'j'k} x_{i'j'k} - M(1 - v_{ij'i'j'}), \quad \forall i \in I, j \in O_i, i' \in I, j' \in O_{i'}, i \neq i' \quad (7)$$

$$c_{ij} \geq y_{kl} + p_{ijk} - M(1 + w_{ijkl} - x_{ijk}), \quad \forall i \in I, j \in O_i, k \in K, l \in L_k \quad (8)$$

$$c_{ij} \leq y_{kl} - \tilde{p}_{kl} + M(2 - w_{ijkl} - x_{ijk}), \quad \forall i \in I, j \in O_i, k \in K, l \in L_k \quad (9)$$

$$v_{ij'i'j'} + v_{i'j'ij} \geq x_{ijk} + x_{i'j'k} - 1, \quad \forall i \in I, j \in O_i, i' \in I, j' \in O_{i'}, i \neq i', k \in K \quad (10)$$

$$v_{ij'i'j'} + v_{i'j'ij} \leq 1, \quad \forall i \in I, j \in O_i, i' \in I, j' \in O_{i'}, i \neq i', k \in K \quad (11)$$

The objective function of this model is to minimize the makespan (1) (denoted as C_{max}). Constraint (2) forces the C_{max} value to be greater than the last operation completed. Constraint (3) ensures that every operation is assigned to a machine. Constraint (4) is the job operation sequence constraint. Constraints (5) and (6) impose maintenance tasks to be scheduled within their corresponding time window. For every pair of operation o_{ij} , $o_{ij'i'j'}$, Constraint (7) imposes that either o_{ij} is scheduled before $o_{i'j'}$ or the opposite in order to prevent operation overlap on the same machine. Similarly, Constraints (8) and (9) prevent overlap between operations and maintenance tasks on the same machine. Constraints (10) and (11) limit the values of $v_{ij'i'j'}$ variables such that, if operations o_{ij} and $o_{ij'i'j'}$ are scheduled on the same machine they cannot be processed simultaneously, i.e. either o_{ij} is processed before or else $o_{ij'i'j'}$ is processed before (this is disjunctive, both cases cannot occur).

3. COMPUTATIONAL STUDY

To evaluate the performance of our proposed MIP, we carry out computational experiments on FJSSP-nfa instances. Previous works on this problem (Gao et al. (2006), Wang and Yu (2010)) used classical FJSSP instances with additional maintenance tasks as test instances, but very few instances are available for this problem in the literature. We first introduce a new benchmark for the FJSSP-nfa using the well-known FJSSP instances. The instances are available on <https://scheduling.cc>.

I :	set of jobs $i \in I$
O_i :	set of operations O_{ij} for job i
K :	set of machines
L_k :	set of maintenance on machine k
A_{ij} :	set of machines available for operation O_{ij}
p_{ijk} :	processing time of operation O_{ij} on machine k
$\tilde{p}_{kl}$:	processing time of maintenance l on machine k
$[t_{kl}^E, t_{kl}^L]$:	time window of maintenance l on machine k

Table 1. Parameters for the FJSSP-nfa.

$c_{ij} (\geq 0)$:	completion time of operation O_{ij}
$y_{kl} (\geq 0)$:	completion time of maintenance l on machine k
x_{ijk}	$= \begin{cases} 1 & \text{if operation } O_{ij} \text{ is assigned to machine } k \\ 0 & \text{otherwise} \end{cases}$
$v_{ij'i'j'}$	$= \begin{cases} 1 & \text{if the processing of operation } O_{i'j'} \text{ starts} \\ & \text{after operation } O_{ij} \text{ finishes} \\ 0 & \text{otherwise} \end{cases}$
w_{ijkl}	$= \begin{cases} 1 & \text{if maintenance } l \text{ on machine } k \text{ starts} \\ & \text{after operation } O_{ij} \text{ finishes} \\ 0 & \text{otherwise} \end{cases}$

Table 2. Variables for the FJSSP-nfa.

Instance	Nb.	O	K	Flexibility	MD
Behnke	60	50-500	20-60	Medium	yes
Brandimarte	15	55-284	4-15	Medium	yes
Chambers	21	100-225	11-18	Low	yes
Dauzère-Pérez	18	196-387	5-10	Medium	yes
Fattahi	20	4-48	2-8	Medium	yes
HurinkE	66	36-300	5-15	None	no
HurinkR	66	36-300	5-15	Low	no
HurinkS	66	36-300	5-15	Medium	no
HurinkV	66	36-300	5-15	High	no
Kacem	4	12-56	5-10	Total	yes

Table 3. FJSSP instance description

3.1 The Proposed Benchmark

The first parameter is the number of maintenance tasks to be scheduled on each machine. Each maintenance task is then defined with a processing time and a time window. The length of the time window must be greater than or equal to the processing time.

We select FJSSP instances from Brandimarte (1993), Kacem et al. (2002), Fattahi et al. (2007), Barnes and Chambers (1995), Dauzère-Pérez and Paulli (1997), Behnke and Geiger (2012) and Hurink et al. (1994) as the basis instances. We solve each instance through a constraint programming model to obtain a reasonably good makespan that is used to estimate the horizon for maintenance tasks. For each FJSSP instance, we generate 5 FJSSP-nfa instances: 4 instances with only one maintenance task per machine whose time window is respectively within the 1st, 2nd, 3rd and 4th quarter of the horizon, as well as a 5th instance with multiple maintenance tasks per machine. For the latter instances, the number of maintenance tasks on a machine is selected randomly with a uniform distribution between 1 and a quarter of the number of potential operations on that machine. In order to generate the time windows, the horizon is equally divided by the number of maintenance tasks. The start and end of each

time window is then chosen randomly within each interval, such that the length of the time window is at least 1. Lastly, the duration of the maintenance tasks is generated randomly with a uniform distribution where the duration of the maintenance tasks is bounded by the length of the corresponding time window.

We used multiple FJSSP benchmarks of 402 instances and generated 5 types of FJSSP-nfa instances (multiple maintenance tasks and one maintenance task with the 1st, 2nd, 3rd, 4th quarter horizon) from each instance for a total of 2010 FJSSP-nfa instances. Table 3 gives a brief description of the FJSSP instances used. The reader is referred to Behnke and Geiger (2012) for more details. In the table, Nb. is the number of instances for this class, $|O|$ the number of operations range, $|K|$ the number of machines range. Flexibility describes the level of flexibility (number of machines available for an operation) of the instance: none if only one machine can process an operation, low if some operations can be processed on multiple machines, medium if most operations can be processed on multiple machines, high if most operations can be processed on most machines and total if any operation can be processed by any machine. Lastly MD describes whether the processing time of an operation is machine-dependant or not.

3.2 Results of The Proposed Formulation

Once we have generated the benchmark, we solved each FJSSP-nfa instance using CPLEX on a Intel(R) Xeon(R) Gold 5120 CPU @ 2.20GHz CPU with a 3600-second timeout. Table 4 summarises the results obtained per class and type of instance. The classes of instances are the classical classes of FJSSP benchmarks and the type of instance is the maintenance task generation procedure (0 for instances with multiple maintenance tasks per machine, 1-4 for instances with only one maintenance task per machine and a reduced horizon). In each class, line “T” in column type combines types 0-4 for the corresponding class. For each type and class, Opt, Feas, NS are respectively the number of instances solved to optimality, the number of instances with a feasible solution and the number of instances with no solution found when the timeout was reached. CPU and Gap are respectively the average time spent solving and the average optimality gap obtained (in %) between the lower bound and the best solution obtained within the time limit.

The first observation is that the number of maintenance tasks and their position in the planing horizon does not have an important impact on the complexity as for each class of instance, results are very close over types 0-4 on average. Type 4 instances obtain the worst average optimality gap on 8 classes of instances. it can be concluded that having maintenance tasks in the last quarter of the planing horizon has negative effect on the MIP solving process. In the other hand, type 1 instances that has one maintenance task in the first quarter obtains the best average optimality gap on 5 classes of instance.

We can also observe that the flexibility can have an important impact on the complexity of an instance as class HurinkE has a total average optimality gap of 15.92% while HurinkR and HurinkV has respectively 30.07%

Class	Type	Opt	Feas	NS	CPU(s)	Gap(%)
Behnke	0	7	23	30	3381.21	62.79
	1	13	21	26	2980.79	58.47
	2	9	24	27	3235.18	61.24
	3	12	20	28	2951.82	60.89
	4	12	21	27	3041.94	61.65
	T	53	109	138	3118.19	61.01
Brandimarte	0	2	9	4	3246.87	60.53
	1	1	13	1	3373.34	51.66
	2	1	10	4	3366.8	58.62
	3	2	9	4	3258.0	60.67
	4	1	10	4	3368.67	63.1
	T	7	51	17	3322.73	58.92
Chambers	0	11	10	0	1971.89	2.46
	1	14	7	0	1685.36	2.62
	2	13	8	0	1968.55	2.08
	3	14	7	0	1809.71	2.41
	4	12	9	0	2299.25	2.88
	T	64	41	0	1946.95	2.49
Dauzère-Pérez	0	0	2	16	3600.14	93.47
	1	0	9	9	3600.15	71.59
	2	0	2	16	3600.16	93.74
	3	0	1	17	3600.41	97.12
	4	0	0	18	3600.27	100.0
	T	0	14	76	3600.23	91.18
Fattahi	0	17	3	0	543.91	1.38
	1	18	2	0	411.94	1.68
	2	17	3	0	545.66	1.94
	3	18	2	0	387.37	1.51
	4	18	2	0	444.99	2.44
	T	88	12	0	466.78	1.79
HurinkE	0	32	31	3	2005.13	14.37
	1	29	37	0	2143.52	16.19
	2	31	33	2	2071.84	16.69
	3	28	35	3	2136.87	18.92
	4	29	34	3	2126.14	21.52
	T	149	170	11	2096.7	17.54
HurinkR	0	22	33	11	2459.46	29.23
	1	22	42	2	2565.91	23.27
	2	22	36	8	2482.73	28.17
	3	22	34	10	2494.55	31.35
	4	21	31	14	2594.24	38.35
	T	109	176	45	2519.38	30.07
HurinkS	0	28	38	0	2265.04	11.67
	1	30	36	0	2079.8	15.34
	2	29	37	0	2112.1	15.08
	3	26	40	0	2228.29	17.72
	4	29	37	0	2195.02	19.77
	T	142	188	0	2176.05	15.92
HurinkV	0	8	40	18	3265.95	46.42
	1	15	36	15	3041.29	43.42
	2	17	31	18	3085.2	43.66
	3	18	30	18	3097.12	46.02
	4	9	37	20	3320.93	48.38
	T	67	174	89	3162.1	45.58
Kacem	0	3	1	0	901.4	7.14
	1	3	1	0	904.83	5.77
	2	4	0	0	661.13	0.0
	3	3	1	0	902.47	5.77
	4	3	1	0	901.93	2.27
	T	16	4	0	854.35	4.19

Table 4. FJSSP-nfa results per class and type of instance

$ O $	$ K $	Opt	Feas	NS	CPU(s)	Gap(%)
9	5	5	0	0	0.01	0.0
12	5	10	0	0	0.05	0.0
15	6	5	0	0	0.28	0.0
15	7	5	0	0	0.52	0.0
18	7	5	0	0	1.05	0.0
21	7	10	0	0	2.34	0.0
24	7	5	0	0	3.31	0.0
29	7	5	0	0	5.51	0.0
30	10	5	0	0	4.67	0.0
32	7	5	0	0	92.63	0.0
36	6	20	0	0	4.71	0.0
36	8	3	2	0	2032.83	1.66
44	8	0	5	0	3600.0	18.35
48	8	0	5	0	3600.01	15.8
49	7	20	0	0	10.37	0.0
50	5	53	47	0	1738.43	9.34
50	20	15	10	0	2108.8	2.85
50	40	20	5	0	1271.27	1.69
50	60	18	7	0	1916.7	1.09
52	4	1	19	0	3490.93	25.85
55	5	9	11	0	2491.63	16.71
55	6	5	0	0	120.58	0.0
56	4	0	20	0	3600.01	31.5
56	10	1	4	0	3407.15	16.76
58	6	2	3	0	2919.71	3.92
60	5	4	16	0	3222.91	18.05
60	6	10	10	0	2003.11	7.92
64	8	20	0	0	225.74	0.0
72	9	20	0	0	98.68	0.0
75	5	0	100	0	3600.01	35.59
90	8	0	5	0	3600.01	25.97

Table 5. FJSSP-nfa results per instance size (less than 100 operations)

45.48%. However, the HurinkS class has a total average optimality gap of 15.92% which is lower than HurinkE. These results show that different levels of flexibility can have both negative and positive impact on the complexity.

In Tables 5 and 6, we compare results between instances of different classes by grouping instances per number of operations and number of machines. Instances are grouped for better readability. Table 5 contains the results for instances having less than 100 operations. On the other hand, Table 6 contains the results for instances having more than 100 operations. $|O|$ is the number of operations and $|K|$ the number of machines. The size of the MIP (number of variables and constraints) depends on both $|O|$ and $|K|$ thus these groups tend to cluster instances per size. Of course, as we mentioned before, the complexity of an instance does not depend only on these two parameters.

Results show that our formulation solves instances with up to 36 operations and 6 machines to optimality in up to 90 seconds on average. Larger instances with up to 100 operations and 13 machines or 50 operations and 60 machines are solved with a reasonably good optimality gap within 3600 seconds for most instances. As shown in Table 5, all instances with less than 100 are solved and a feasible solution is obtained. Most larger instances with up to 225 operations and 16 machines are more tedious to solve even though some instances (150 operations with 11, 12 and 13 machines and 225 operations with 16, 17 and 18 machines) are much easier to solve due to the structure

$ O $	$ K $	Opt	Feas	NS	CPU(s)	Gap(%)
100	5	0	121	4	3600.01	54.77
100	10	301	59	0	1047.48	1.14
100	11	9	1	0	610.97	0.54
100	12	14	1	0	829.85	0.09
100	13	10	0	0	643.88	0.0
100	20	0	25	0	3600.07	46.13
100	40	0	25	0	3600.04	41.71
100	60	0	25	0	3600.12	44.54
106	4	0	5	0	3600.02	63.25
150	8	0	5	0	3600.03	44.69
150	10	4	101	0	3534.54	23.42
150	11	6	4	0	2299.93	1.58
150	12	5	10	0	2697.54	3.61
150	13	6	4	0	2385.74	3.6
179	5	0	5	0	3600.09	66.2
193	10	0	5	0	3600.1	60.94
196	5	0	11	19	3600.09	79.2
200	10	0	69	31	3600.07	54.9
225	10	0	5	0	3600.07	64.06
225	15	5	67	28	3498.32	33.97
225	16	3	7	0	2708.7	6.5
225	17	7	8	0	2516.78	3.62
225	18	4	6	0	2727.53	2.94
231	10	0	0	5	3600.09	100.0
240	10	0	1	4	3600.04	93.21
240	15	0	1	4	3600.07	92.46
250	20	0	2	23	3600.14	99.67
250	40	0	3	22	3600.3	99.38
250	60	0	7	18	3600.18	95.03
277	15	0	5	0	3600.12	60.51
284	15	0	1	4	3600.04	91.34
293	8	0	2	28	3600.2	96.08
300	10	0	42	58	3600.2	80.89
300	15	0	36	24	3600.15	56.79
387	10	0	1	29	3600.39	98.27
500	20	0	0	25	3600.54	100.0
500	40	0	0	25	3600.62	100.0
500	60	0	0	25	3319.48	100.0

Table 6. FJSSP-nfa results per instance size (more than 100 operations)

of the instance. The largest instances are too difficult to solve for the proposed formulation as we can only obtain solutions with a very high gap, or no solution at all within the time limit.

4. CONCLUSION

In this study, we considered a flexible job shop scheduling problem under availability constraints. A MIP model has been proposed to solve the FJSSP-nfa. In order to test the proposed model, we introduced the first large benchmark for this problem and solved it. The experimental results shows the effectiveness of the model on small- and medium-sized instances.

As future research, we consider multiple perspectives. The solving process can be enhanced by designing good lower bounds or valid inequalities. Constraint programming is also a promising perspective with the recent works in this field.

REFERENCES

- Aggoune, R. (2004). Two-job shop scheduling problems with availability constraints. In *ICAPS*, 253–259.
- Al-Hinai, N. and ElMekkawy, T.Y. (2011). Robust and stable flexible job shop scheduling with random machine breakdowns using a hybrid genetic algorithm. *International Journal of Production Economics*, 132(2), 279–291.
- Barnes, J.W. and Chambers, J.B. (1995). Solving the job shop scheduling problem with tabu search. *IIE transactions*, 27(2), 257–263.
- Behnke, D. and Geiger, M.J. (2012). Test instances for the flexible job shop scheduling problem with work centers. *Arbeitspapier/Research Paper/Helmut-Schmidt-Universität, Lehrstuhl für Betriebswirtschaftslehre, insbes. Logistik-Management*.
- Brandimarte, P. (1993). Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations research*, 41(3), 157–183.
- Brucker, P. and Schlie, R. (1990). Job-shop scheduling with multi-purpose machines. *Computing*, 45(4), 369–375.
- Dauzère-Pérès, S. and Paulli, J. (1997). An integrated approach for modeling and solving the general multiprocessor job-shop scheduling problem using tabu search. *Annals of Operations Research*, 70, 281–306.
- El Khoukhi, F., Boukachour, J., and Alaoui, A.E.H. (2017). The “dual-ants colony”: A novel hybrid approach for the flexible job shop scheduling problem with preventive maintenance. *Computers & Industrial Engineering*, 106, 236–255.
- Fattahi, P., Saidi Mehrabad, M., and Jolai, F. (2007). Mathematical modeling and heuristic approaches to flexible job shop scheduling problems. *Journal of intelligent manufacturing*, 18(3), 331–342.
- Gao, J., Gen, M., and Sun, L. (2006). Scheduling jobs and maintenances in flexible job shop with a hybrid genetic algorithm. *Journal of Intelligent Manufacturing*, 17(4), 493–507.
- Hurink, J., Jurisch, B., and Thole, M. (1994). Tabu search for the job-shop scheduling problem with multi-purpose machines. *Operations-Research-Spektrum*, 15(4), 205–215.
- Jing, Z., Hua, J., and Yi, Z. (2017). Multi-objective integrated optimization problem of preventive maintenance planning and flexible job-shop scheduling. 137–141. Springer.
- Kacem, I., Hammadi, S., and Borne, P. (2002). Pareto-optimality approach for flexible job-shop scheduling problems: hybridization of evolutionary algorithms and fuzzy logic. *Mathematics and computers in simulation*, 60(3–5), 245–276.
- Kress, D., Müller, D., and Nossack, J. (2019). A worker constrained flexible job shop scheduling problem with sequence-dependent setup times. *OR Spectrum*, 41(1), 179–217.
- Li, J.Q. and Pan, Q.K. (2012). Chemical-reaction optimization for flexible job-shop scheduling problems with maintenance activity. *Applied soft computing*, 12(9), 2896–2912.
- Li, J.Q., Pan, Q.K., and Tasgetiren, M.F. (2014). A discrete artificial bee colony algorithm for the multi-objective flexible job-shop scheduling problem with maintenance activities. *Applied Mathematical Modelling*, 38(3), 1111–1132.
- Pezzella, F., Morganti, G., and Ciaschetti, G. (2008). A genetic algorithm for the flexible job-shop scheduling problem. *Computers & operations research*, 35(10), 3202–3212.
- Rajkumar, M., Asokan, P., Anilkumar, N., and Page, T. (2011). A grasp algorithm for flexible job-shop scheduling problem with limited resource constraints. *International Journal of Production Research*, 49(8), 2409–2423.
- Rossi, A. (2014). Flexible job shop scheduling with sequence-dependent setup and transportation times by ant colony with reinforced pheromone relationships. *International Journal of Production Economics*, 153, 253–267.
- Saidi-Mehrabad, M. and Fattahi, P. (2007). Flexible job shop scheduling with tabu search algorithms. *The international journal of Advanced Manufacturing technology*, 32(5), 563–570.
- Taillard, E. (1993). Benchmarks for basic scheduling problems. *European Journal of Operational Research*, 64(2), 278–285.
- Wang, H., Sheng, B., Lu, Q., Yin, X., Zhao, F., Lu, X., Luo, R., and Fu, G. (2021). A novel multi-objective optimization algorithm for the integrated scheduling of flexible job shops considering preventive maintenance activities and transportation processes. *Soft Computing*, 25, 2863–2889.
- Wang, L., Zhou, G., Xu, Y., Wang, S., and Liu, M. (2012). An effective artificial bee colony algorithm for the flexible job-shop scheduling problem. *The International Journal of Advanced Manufacturing Technology*, 60(1), 303–315.
- Wang, S. and Yu, J. (2010). An effective heuristic for flexible job-shop scheduling problem with maintenance activities. *Computers & Industrial Engineering*, 59(3), 436–447.
- Xing, L.N., Chen, Y.W., Wang, P., Zhao, Q.S., and Xiong, J. (2010). A knowledge-based ant colony optimization for flexible job shop scheduling problems. *Applied Soft Computing*, 10(3), 888–896.
- Xiong, J., Xing, L.n., and Chen, Y.w. (2013). Robust scheduling for multi-objective flexible job-shop problems with random machine breakdowns. *International Journal of Production Economics*, 141(1), 112–126.
- Zhang, G., Shao, X., Li, P., and Gao, L. (2009). An effective hybrid particle swarm optimization algorithm for multi-objective flexible job-shop scheduling problem. *Computers & Industrial Engineering*, 56(4), 1309–1318.
- Ziaee, M. (2014). An efficient heuristic algorithm for flexible job shop scheduling with maintenance constraints. *Applied Mathematics and Sciences: An International Journal (MathSJ)*, 1(1), 19–31.